

NAME: ASHIRA BRIDGET SUSAN S

REG. NO.: 192524316

COURSE CODE: CSA0201

COURSE NAME: C PROGRAMMING

ASSESSMENT: CO4 AT2 (CASE STUDY QUESTIONS)

CASE STUDY 1: HOSPITAL PATIENT MANAGEMENT SYSTEM

Code:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
typedef struct {
```

```
    int id;
```

```
    char name[50];
```

```
    int age;
```

```
    char bg[5];
```

```
    char phone[15];
```

```
    char diag[100];
```

```
} Patient;
```

```
void show(const Patient *p);
```

```
void update(Patient *p, const char *new_diag);
```

```
int main() {
```

```
    Patient p;
```

```
    char new_diag[100];
```

```
    printf("Enter ID: ");
```

```
    scanf("%d", &p.id);
```

```
    getchar();
```

```
    printf("Enter Name: ");
```

```

fgets(p.name, sizeof(p.name), stdin);
p.name[strcspn(p.name, "\n")] = '\0';
printf("Enter Age: ");
scanf("%d", &p.age);
getchar();
printf("Enter Blood Group: ");
fgets(p.bg, sizeof(p.bg), stdin);
p.bg[strcspn(p.bg, "\n")] = '\0';
printf("Enter Phone: ");
fgets(p.phone, sizeof(p.phone), stdin);
p.phone[strcspn(p.phone, "\n")] = '\0';
printf("Enter Diagnosis: ");
fgets(p.diag, sizeof(p.diag), stdin);
p.diag[strcspn(p.diag, "\n")] = '\0';
show(&p);

printf("Enter New Diagnosis: ");
fgets(new_diag, sizeof(new_diag), stdin);
new_diag[strcspn(new_diag, "\n")] = '\0';
update(&p, new_diag);
show(&p);
return 0;
}

void show(const Patient *p) {
    if (p != NULL) {
        printf("ID: %d\n", p->id);
        printf("Name: %s\n", p->name);
        printf("Age: %d\n", p->age);
        printf("Blood Group: %s\n", p->bg);
    }
}

```

```

    printf("Phone: %s\n", p->phone);
    printf("Diagnosis: %s\n", p->diag);
}
}

void update(Patient *p, const char *new_diag) {
    if (p != NULL) {
        strncpy(p->diag, new_diag, sizeof(p->diag) - 1);
        p->diag[sizeof(p->diag) - 1] = '\0';
    }
}

```

Analysis & Explanation

1. How Records Are Passed to Functions Using Pointers

- **Address Operator (&):** We pass the memory address of the patient structure (&p) into the function instead of passing the entire structure data.
- **Pointer Parameter (Patient *p):** The function takes this memory address as a pointer parameter.
- **Arrow Operator (->):** Inside the function, we use -> (like p->diag) to directly read or change the values at that memory location.

2. Why Structures and Structure Pointers Are Used

- **No Extra Memory Copying:** Passing a full structure by value copies all its fields into new memory. Passing by pointer only copies a single memory address (4 or 8 bytes), saving RAM and CPU speed when handling thousands of patient records.

3. Consequences of Modifying a Structure Through a Pointer

- **Direct Changes:** Any change made inside update(&p, new_diag) updates the original data directly in memory.
- **Risk of Accidental Data Corruption:** If a pointer modifies data by mistake, the original data is changed permanently.
- **Prevention (const):** We use const Patient *p in read-only functions like show() so the compiler stops any accidental changes to the patient's data.

CASE STUDY 2: IOT SENSOR GATEWAY

Code:

```
#include <stdio.h>

typedef enum {
    TEMP,
    HUMID,
    PRESS,
    VIB
} Type;

typedef union {
    float temp;
    float humid;
    float press;
    float vib;
} Data;

typedef struct {
    Type type;
    Data data;
} Sensor;

int main() {
    Sensor s;

    printf("Select type (0: Temp, 1: Humid, 2: Press, 3: Vib): ");
    scanf("%d", (int*)&s.type);

    if (s.type == TEMP) {
        printf("Enter Temp: ");
        scanf("%f", &s.data.temp);
    } else if (s.type == HUMID) {
```

```

        printf("Enter Humidity: ");
        scanf("%f", &s.data.humid);
    } else if (s.type == PRESS) {
        printf("Enter Pressure: ");
        scanf("%f", &s.data.press);
    } else if (s.type == VIB) {
        printf("Enter Vibration: ");
        scanf("%f", &s.data.vib);
    }

    if (s.type == TEMP) {
        printf("Read Temp: %.2f\n", s.data.temp);
    } else if (s.type == HUMID) {
        printf("Read Humidity: %.2f\n", s.data.humid);
    } else if (s.type == PRESS) {
        printf("Read Pressure: %.2f\n", s.data.press);
    } else if (s.type == VIB) {
        printf("Read Vibration: %.2f\n", s.data.vib);
    }

    printf("Union Size: %lu bytes\n", sizeof(Data));
    return 0;
}

```

Analysis & Justification

1. Union vs Structure Choice

- A union should be used for the sensor payload because the system receives only one type of reading at a time.
- A structure would allocate memory for all four values at once (4×4 bytes = 16 bytes). A union shares the same memory space for all variables, taking only 4 bytes.

2. Memory Allocation Difference

- Structure: Memory size = Sum of all members ($4 + 4 + 4 + 4 = 16$ bytes).

- Union: Memory size = Size of the largest member (4 bytes).
- Result: Using a union saves 75% memory, which minimizes RAM usage in an IoT gateway.

3. Situation Where Wrong Choice Produces Incorrect Results

- If you store a temperature value into the union, and then try to read it as humidity or pressure without updating it, you will get corrupted/garbage data.
- Example: Writing `s.data.temp = 36.5` and then reading `s.data.press` without checking `s.type` will return invalid data because both variables share the exact same memory address.

CASE STUDY 3: BANKING TRANSACTION PROCESSOR

```
#include <stdio.h>

void process_tx(double amt) {
    auto double fee = 2.50;
    static int tx_count = 0;
    static double balance = 1000.0;

    tx_count++;
    balance += (amt - fee);

    printf("Tx #%d | Amount: %.2f | Fee: %.2f | Balance: %.2f\n",
        tx_count, amt, fee, balance);
}

int main() {
    double amt;

    printf("Enter transaction 1 amount: ");
```

```

scanf("%lf", &amt);

process_tx(amt);

printf("Enter transaction 2 amount: ");
scanf("%lf", &amt);
process_tx(amt);

return 0;
}

```

Analysis & Storage Class Explanation

1. Lifetime, Scope, and Visibility of Each Variable

Variable	Storage Class	Scope	Lifetime	Visibility
fee	auto	Local (inside process_tx)	Destroys when function ends	Inside process_tx only
tx_count	static	Local (inside process_tx)	Entire program runtime	Inside process_tx only
balance	static	Local (inside process_tx)	Entire program runtime	Inside process_tx only

2. Why Each Storage Class Is Used

- **static for Counters & Balances:** A static variable keeps its value in memory even after the function finishes. When process_tx() is called a second time, tx_count remembers its previous value and increments to 2 instead of resetting to 0.
- **auto for Temporary Data:** Local variables default to auto. fee is created when the function starts and destroyed when it ends, saving memory.
- **extern for Global Access:** If balance needed to be accessed directly by other source files in a multi-file program, it would be declared globally as extern.

CASE STUDY 4: UNIVERSITY STUDENT DATABASE

Code:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
typedef struct {  
    char city[30];  
} Addr;
```

```
typedef struct {  
    float marks;  
    float att;  
} Acad;
```

```
typedef struct {  
    int id;  
    char name[50];  
    Addr addr;  
    Acad acad;  
} Student;
```

```
void update(Student *s, float m, float a);
```

```
void show(const Student *s);
```

```
int main() {  
    Student s;  
    float new_m, new_a;  
    printf("Enter ID: ");  
    scanf("%d", &s.id);  
    getchar();  
    printf("Enter Name: ");  
    fgets(s.name, sizeof(s.name), stdin);
```

```

s.name[strcspn(s.name, "\n")] = '\0';
printf("Enter City: ");
fgets(s.addr.city, sizeof(s.addr.city), stdin);
s.addr.city[strcspn(s.addr.city, "\n")] = '\0';
printf("Enter Marks: ");
scanf("%f", &s.acad.marks);
printf("Enter Attendance: ");
scanf("%f", &s.acad.att);
show(&s);

printf("Enter New Marks: ");
scanf("%f", &new_m);
printf("Enter New Attendance: ");
scanf("%f", &new_a);
update(&s, new_m, new_a);
show(&s);
return 0;
}

void update(Student *s, float m, float a) {
    if (s != NULL) {
        s->acad.marks = m;
        s->acad.att = a;
    }
}

void show(const Student *s) {
    if (s != NULL) {
        printf("ID: %d\n", s->id);
        printf("Name: %s\n", s->name);
        printf("City: %s\n", s->addr.city);
    }
}

```

```

printf("Marks: %.1f\n", s->acad.marks);

printf("Attendance: %.1f%%\n", s->acad.att);

}

}

```

Analysis & Explanation

1. Difference Between . and -> Operators

- Dot Operator (.): Used when accessing members using a normal structure variable directly (e.g., s.id or s.addr.city).
- Arrow Operator (->): Used when accessing members using a pointer variable to a structure (e.g., s->acad.marks). s->acad.marks is a shortcut for (*s).acad.marks.

2. How Improper Pointer Handling Corrupts Student Records

- Null Pointers: Accessing an uninitialized or NULL pointer (e.g., calling update(NULL, 90, 95) without checking) causes a segmentation fault and crashes the program.
- Wild/Dangling Pointers: If a pointer points to an invalid or freed memory location, assigning values through s->acad.marks overwrites random memory, corrupting other active student records.

CASE STUDY 5: EMBEDDED MEDICAL DEVICE

Code:

```

#include <stdio.h>

void run_monitor() {
    register int i;
    int limit;
    int pulse = 72;

    printf("Enter loop limit: ");
    scanf("%d", &limit);

    for (i = 0; i < limit; i++) {

```

```

        pulse++;
    }
    printf("Updated Pulse: %d\n", pulse);
}

int main() {
    run_monitor();
    return 0;
}

```

Analysis & Justification

1. Does register Guarantee Faster Execution?

- No, it does not guarantee it. register is only a hint to the compiler to store a variable in a high-speed CPU register instead of RAM.
- Modern optimizing compilers decide whether to place a variable in a register based on register availability, regardless of the register keyword.

2. Restrictions of register vs auto

- Taking Address (&): You cannot take the memory address of a register variable using the & operator (e.g., &i will cause a compiler error) because registers do not have a RAM memory address. An auto variable resides in RAM, so taking its address (&var) is fully allowed.
- Storage Space: register can only be used on simple scalar types (like int, char, pointer) that fit into a CPU register. Complex structures or large arrays cannot use register.

CASE STUDY 6: MULTI-MODULE RAILWAY RESERVATION SYSTEM

Code:

```

#include <stdio.h>

int total_res = 100;

static int module_res = 10;

void make_booking() {
    total_res++;
    module_res++;
}

```

```
printf("Booking done! Total: %d, Module: %d\n", total_res, module_res);  
}
```

```
int main() {  
    printf("Initial Total: %d, Module: %d\n", total_res, module_res);  
    make_booking();  
    return 0;  
}
```

Analysis & Explanation

1. How extern and static Meet the Requirements

- **extern (Global Accessibility):** Declaring `total_res` as a global variable allows other source files (like ticket booking or payment processing) to access and modify the total reservations using the `extern` keyword (e.g., `extern int total_res;`).
- **static (File Privacy):** Declaring `module_res` as a static global limits its scope only to its own source file. Other files cannot see or modify it, keeping the counter private to that module.

2. What Happens if extern or static is Used Incorrectly?

- **Incorrect static:** If `total_res` is wrongly declared as `static`, other modules cannot access it. The compiler will give an "unresolved external symbol" error during linking.
- **Incorrect extern (or missing static):** If `module_res` is declared without `static`, any other file can access or modify it, leading to unintended changes or variable name conflicts across files.

CASE STUDY 7: VEHICLE DIAGNOSTIC SYSTEM

Code:

```
#include <stdio.h>  
  
typedef enum {  
    MODE_TEMP,  
    MODE_VOLT,
```

```

    MODE_ERR
} Mode;

typedef union {
    float temp;
    float volt;
    int err;
} DiagData;

typedef struct {
    Mode mode;
    DiagData data;
} Packet;

int main() {
    Packet p;

    printf("Select mode (0: Temp, 1: Volt, 2: Error): ");
    scanf("%d", (int*)&p.mode);

    if (p.mode == MODE_TEMP) {
        printf("Enter Engine Temp: ");
        scanf("%f", &p.data.temp);
    } else if (p.mode == MODE_VOLT) {
        printf("Enter Battery Voltage: ");
        scanf("%f", &p.data.volt);
    } else if (p.mode == MODE_ERR) {
        printf("Enter Error Code: ");
        scanf("%d", &p.data.err);
    }

    if (p.mode == MODE_TEMP) {
        printf("Read Engine Temp: %.1f C\n", p.data.temp);
    }
}

```

```

    } else if (p.mode == MODE_VOLT) {
        printf("Read Battery Voltage: %.1f V\n", p.data.volt);
    } else if (p.mode == MODE_ERR) {
        printf("Read Error Code: %d\n", p.data.err);
    }

    printf("Packet Memory Size: %lu bytes\n", sizeof(Packet));

    return 0;
}

```

Analysis & Explanation

1. Memory Layout (Structure + Union)

- Structure: Holds the active mode enum (Mode mode) and the union (DiagData data).
- Union: Contains temp, volt, and err. All three variables share the exact same memory space (4 bytes, which is the size of a float or int).
- Total Memory: 4 bytes (for enum) + 4 bytes (for union) = 8 bytes total memory space.

2. Accessing Different Union Members (Unexpected Results)

- If you set p.mode = MODE_VOLT and store p.data.volt = 12.5, but later try to read p.data.err, the output will be corrupted garbage data.
- Reason: In a union, writing a value overwrites whatever was stored previously because all members share the same memory bytes. Storing a float writes floating-point bits, which cannot be read directly as an int error code.

CASE STUDY 8: LIBRARY MANAGEMENT SYSTEM

Code:

```

#include <stdio.h>

#include <string.h>

typedef struct {
    int id;
    char title[50];
    char author[50];

```

```
float price;

int avail;

} Book;

void show(const Book *b);

void update_price(Book *b, float new_price);

int main() {

    Book b;

    printf("Enter Book ID: ");

    scanf("%d", &b.id);

    getchar();

    printf("Enter Title: ");

    fgets(b.title, sizeof(b.title), stdin);

    b.title[strcspn(b.title, "\n")] = '\0';

    printf("Enter Author: ");

    fgets(b.author, sizeof(b.author), stdin);

    b.author[strcspn(b.author, "\n")] = '\0';

    printf("Enter Price: ");

    scanf("%f", &b.price);

    printf("Enter Availability (1 for Yes, 0 for No): ");

    scanf("%d", &b.avail);

    show(&b);

    printf("Enter New Price: ");

    scanf("%f", &b.price);

    update_price(&b, b.price);

    show(&b);

    return 0;
```

```

}

void show(const Book *b) {
    if (b != NULL) {
        printf("ID: %d\n", b->id);
        printf("Title: %s\n", b->title);
        printf("Author: %s\n", b->author);
        printf("Price: $%.2f\n", b->price);
        printf("Available: %s\n", b->avail ? "Yes" : "No");
    }
}

void update_price(Book *b, float new_price) {
    if (b != NULL) {
        b->price = new_price;
    }
}

```

Analysis & Justification

1. Passing Structures by Value vs. Passing Structure Addresses (Pointers)

Feature	Passing by Value	Passing by Pointer (Address)
Memory Allocation	Creates a complete copy of the structure on the stack.	Passes only a small memory address (4 or 8 bytes).
Execution Speed	Slow for large structures with hundreds of bytes.	Extremely fast because no data is copied.
Data Modification	Changes stay local inside the function; original record is unchanged.	Changes update the original record directly in memory.

2. Recommendation

- Better Approach: Passing structure addresses (pointers) is the recommended approach.

- Justification: Since a library holds thousands of book records (each taking over 100 bytes for title, author, etc.), passing by value causes massive stack memory copying during searches, sorts, and updates. Passing structure addresses (Book *b) optimizes CPU performance and drastically minimizes memory consumption.

CASE STUDY 9: OPERATING SYSTEM PROCESS TABLE

Code:

```
#include <stdio.h>

#include <string.h>

typedef struct {

    int pid;

    char state[10];

    int priority;

    float cpu;

} Process;

static int total_procs = 0;

int get_next_pid() {

    static int pid_counter = 1000;

    return ++pid_counter;

}

void create_proc(Process *p, const char *state, int priority, float cpu) {

    p->pid = get_next_pid();

    strncpy(p->state, state, sizeof(p->state) - 1);

    p->state[sizeof(p->state) - 1] = '\0';

    p->priority = priority;

    p->cpu = cpu;

    total_procs++;

}

void show_proc(const Process *p) {
```

```

printf("PID: %d | State: %s | Priority: %d | CPU: %.1f%%\n",
      p->pid, p->state, p->priority, p->cpu);
}

int main() {
    Process p1, p2;
    create_proc(&p1, "RUNNING", 1, 12.5);
    create_proc(&p2, "READY", 2, 0.0);
    show_proc(&p1);
    show_proc(&p2);
    printf("Total Active Processes: %d\n", total_procs);
    return 0;
}

```

Analysis & Explanation

1. Static Local vs. Static Global Variables

Feature	Static Local (pid_counter)	Static Global (total_procs)
Scope	Restricted only inside get_next_pid() function.	Accessible by all functions inside this source file.
Lifetime	Retains value across function calls for the entire program execution.	Retains value for the entire program execution.
Linkage	Internal (no linkage outside the function).	Internal linkage (private to this file, hidden from other files).

2. Scope, Lifetime, and Linkage Summary

- **pid_counter (Static Local):** Holds the auto-incrementing process ID counter across function calls. Its scope is limited to get_next_pid(), its lifetime is program runtime, and it has no linkage.
- **total_procs (Static Global):** Tracks total created processes across the file. Its scope is file-wide, its lifetime is program runtime, and its linkage is internal (cannot be accessed by other .c files).

CASE STUDY 10: INTEGRATED SMART FACTORY CONTROLLER

Code:

```
#include <stdio.h>

#include <string.h>


int global_factory_id = 5001;

typedef union {

    float temp;

    float press;

} Sensor;

typedef struct {

    int id;

    int state;

    Sensor data;

} Machine;

int get_security_token() {

    static int token = 100;

    return ++token;

}

void process_machine(Machine *m) {

    register int i;

    auto float threshold = 80.0;

    for (i = 0; i < 1; i++) {

        if (m->data.temp > threshold) {

            m->state = 0;

        } else {

            m->state = 1;

        }

    }

}
```

```

}

int main() {
    Machine m1;
    m1.id = 101;
    printf("Enter Machine Temperature: ");
    scanf("%f", &m1.data.temp);
    process_machine(&m1);
    printf("Factory ID: %d\n", global_factory_id);
    printf("Machine ID: %d\n", m1.id);
    printf("Security Token: %d\n", get_security_token());
    printf("Sensor Temp: %.1f C\n", m1.data.temp);
    printf("Machine Status: %s\n", m1.state ? "OK" : "OVERHEAT WARNING");
    return 0;
}

```

Design Evaluation & Justifications

1. Why Each Feature Is Used

- Structures (Machine): Used to bundle related properties (Machine ID, status, and sensor readings) together into a single organized entity.
- Unions (Sensor): Used to store temperature or pressure in the exact same memory location, saving RAM since only one sensor reading is taken at a time.
- Pointers (Machine *m): Used in process_machine() to avoid copying the structure memory onto the stack and to allow direct updates to machine status.
- auto (threshold): Used for standard temporary function variables that are automatically cleaned up when the function exits.
- static (token): Used inside get_security_token() to persist the token counter value across function calls without exposing it to the rest of the program.
- extern (global_factory_id): Used to allow global configuration data to be accessed across multiple module files in the system.
- register (i): Used as a hint to place fast loop counters into CPU registers for minimal latency.

2. Potential Implementation Risks

1. Union Data Overwriting: Reading the wrong member of a union (e.g., reading `press` when `temp` was written) will return corrupted, invalid numerical data.
2. Pointer Corruption (Null/Wild Pointers): Passing an uninitialized or NULL structure pointer to `process_machine()` will cause a segmentation fault and crash the factory controller.